SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. *Introduction:*

MedFlow *is an intelligent agentic solution designed to transform fragmented and manual medical intake and triage processes into a seamless automated workflow. By utilizing Z.AI's GLM as a central reasoning engine, the system eliminates the inefficiencies of human led data coordination and unstructured medical documentation.*

1.1. *Purpose:*

The system analysis document highlights the technical scope and design related decisions behind the ***MedFlow*** *development. It outlines how the agentic engine handles multi-step medical reasoning, dynamic tool orchestration, and the generation of actionable clinical outputs to support healthcare providers.*

So, the key element of system that has been covered are as followed:

1. ***Architecture****:*

a. ***MedFlow*** *is built as a* ***cloud-native platform*** *utilizing a* ***microservices architecture*** *to ensure modularity and independent scaling of medical agents.*
b. *The system is deployed on* ***AWS****, where specific services such as the* ***Triage Agent, Insurance Validator, and Scheduler Agent*** *operate as independent microservices.*

2. ***Data Flows:***

a. *Unstructured patient data (voice notes or text) flows from the* ***Client Dashboard*** *to the* ***Backend API****.*
b. *The* ***GLM Service Layer*** *processes this data, extracting clinical entities which are then stored in a* ***PostgreSQL database*** *via* ***AWS RDS****.*
c. *Finally, the system returns a* ***structured medical report*** *and suggested actions back to the medical staff interface*

3. ***Model Process:***

a. *The model demonstrates an* ***end-to-end agentic workflow****: from initial* ***Symptom Extraction*** *and* ***Clinical Guideline Cross-Checking*** *to final* ***Appointment Scheduling****.*
b. *It highlights how the* ***GLM Orchestrator*** *manages transitions between these stages and handles real-world constraints like missing patient history.*

4. ***Role of Reference:***

This document serves as a technical reference for developers and testers to understand the high-level architecture and agentic logic of the project.

© UMHackathon 2026

1.2. Background:

1. **Current State**: Many medical organizations rely on fragmented, manual, and unstructured workflows to complete everyday patient intake tasks.
2. **Identified Problem**: These workflows require heavy human interpretation and coordination across tools, resulting in inefficiencies and errors.
3. **New Capabilities**: MedFlow introduces a "stateful and adaptive workflow engine" where the GLM acts as the central reasoning core to automate these manual processes.

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Clinical Staff	Input unstructured patient data and manage the triage dashboard.	Significant reduction in manual data entry and coordination effort.
Doctors	Review AI-generated reasoning paths and approve triage actions.	Clear, structured patient profiles and logical clinical justifications.
Development Team	Build and maintain the agentic orchestrator and microservices.	Modular codebase with clear API contracts between the GLM and tools.
QA Team	Validate system behaviour across edge cases like missing patient data.	Defined coverage of failure states and prompt-response accuracy.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Mobile App (iOS & Android)*
Architecture	*[e.g. Cloud-Server with Microservices]*

MedFlow is primarily structured as a client-server system where a React-based dashboard communicates with a Backend API layer. The system is deployed on Amazon Web Services (AWS), using PostgreSQL hosted in AWS RDS for persistent storage

2.1.2 GLM as Service Layer

The Z.AI GLM is integrated as a central service layer that handles the "thinking" of the system.

2.1.3 Dependency Diagram

a. Prompt Construction: Prompts are engineered using a multi-step agentic strategy, where patient symptoms are first structured before being sent to the Z.AI GLM API for clinical reasoning.
b. Context Window: The context window includes extracted clinical entities, current triage status, and relevant patient medical history retrieved from the hospital CRM.
c. Response Parsing: The system receives raw GLM responses, validates them against a clinical JSON schema, and passes the structured data to the Scheduler or Insurance agents.
d. Token Management: Inputs are truncated or chunked at the 1,500-token limit before reaching the GLM to ensure efficiency and avoid processing failures.
e. Major API Calls: The Client Dashboard calls the Amazon API Gateway to trigger the Triage Orchestration Service.
f. Component Interaction: The Scheduler Service communicates with Amazon RDS to ensure the persistence of booked appointment data.
g. Distributed Deployment: MedFlow is deployed across multiple microservices on AWS to allow independent scaling of the Triage and Billing modules

© UMHackathon 2026

2.2.2. Sequence Diagram

- Step 1: Input Submission: The Clinical Staff records a patient's voice note on the React Dashboard.
- Step 2: Orchestration: The Backend API sends the raw audio to the Z.AI GLM Service Layer for multi-step reasoning.
- Step 3: Clinical Extraction: The GLM extracts symptoms and patient history, identifying medical urgency.
- Step 4: Tool Interaction: The GLM Orchestrator triggers the Insurance Validator to verify coverage and the Scheduler Agent to find open slots.
- Step 5: Verification: The Doctor reviews the AI's "Reasoning Canvas" and approves the generated triage report.
- Step 6: Confirmation: The system stores the final record in PostgreSQL and sends a confirmation notification to the patient.

2.2. Technological Stack

2.2.1 Frontend: React (with Tailwind CSS) – Chosen for its rapid prototyping capabilities and modular component-based architecture.
2.2.2 Backend: Java (Spring Boot) – Provides a robust, type-safe environment for managing complex agentic workflows and multi-step reasoning.
2.2.3 Database: PostgreSQL – Used as the relational database for persistent storage of patient profiles, clinical history, and triage logs.
2.2.4 Cloud/Deployment: AWS EC2 (Server) for application hosting and AWS RDS (PostgreSQL) for managed database services.

2.3. Key Data Flows

2.3.1.1. Data Flow Diagram (DFD)

- Process 1.0 (Ingestion): Raw unstructured data (text/voice) moves from the Patient/Nurse to the Backend Gateway.
- Process 2.0 (Agentic Reasoning): The data is passed to the Z.AI GLM, where it is transformed into a Structured JSON Object.
- Data Store (Clinical History): Parsed clinical data is saved in the PostgreSQL RDS database.
- Process 3.0 (Orchestration): Validated diagnostic data is sent to External APIs (Insurance/EHR) for cross-referencing.
- Process 4.0 (Action Generation): The final scheduled appointment and triage report are output to the Medical Staff Dashboard.

© UMHackathon 2026

2.3.2. **Normalized Database Schema**

- Patients Table: Stores basic demographic info (Patient_ID as PK).
- Symptoms Table: A junction table resolving the M:N relationship between Patients and Clinical Diagnoses.
- Reasoning Logs Table: Stores the GLM's specific multi-step reasoning paths for audit trails.
- Appointments Table: Stores finalized dates, times, and assigned specialist IDs.

3. Functional Requirements & Scope

3.1. Minimum Viable Product:

#	*Features*	*Description*
1	Intelligent Intake	GLM extracts symptoms from unstructured voice/text into a structured profile.
2	Agentic Triage	Multi-step reasoning to assign patient priority based on clinical guidelines.
3	Auto-Scheduling	Dynamic orchestration with calendar APIs to book appointment slots.

© UMHackathon 2026

3.2. *Non-Functional Requirements (NFRs)*

Quality (e.g.)	***Requirements (e.g.)***	***Implementation (e.g)***
Scalability	The system must handle sudden spikes in patient triage requests during peak clinical hours.	Use AWS Auto Scaling Groups (ASG) for servers and ELB for intelligent traffic distribution.
Reliability	Critical triage paths must have 99% uptime to prevent delays in emergency medical processing.	Successful triage completion triggers a persistent Database INSERT to prevent data loss.
Maintainability	The Triage, Insurance, and Scheduling agents must be separated and independently deployable	Utilize independent CI/CD pipelines for each microservice with unique environment configurations.
Token Latency	GLM reasoning responses for triage priority must return within 3 seconds under normal load.	Implement a 5s timeout for asynchronous API calls and display a "Reasoning" state to the user.
Cost Efficiency	Token expenditure per patient session must stay within the defined project budget	Clinical prompts use fixed lengths; repeating diagnostic queries are cached for 24 hours.

3.3. *Out of Scope / Future Enhancements*

a. **In-App Chat**: Real-time communication between doctor and patient during a visit.
b. **Voucher Systems**: Implementation of medical discounts or promotional insurance codes.
c. **Historical Analytics**: Long-term patient health trend monitoring over several years.

© UMHackathon 2026

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1 ***Grayscale Rollout & A/B Testing****:* New triage logic variants are initially released to 5% of clinic workstations to monitor for accuracy before a 100% rollout.

4.1.2 ***Emergency Rollback (ER) & Golden Release****:* If high-severity clinical hallucinations are detected, the automated pipeline triggers an Emergency Rollback (ER) to the Golden Version.

4.1.3 ***Priority Matrix****:*

4.1.3.1 P1 Critical is defined as any instance where the GLM reasoning engine fails to produce structured output for more than 1% of total requests *Monitoring:*

4.2. Assumptions:

It is assumed that clinical staff have a stable internet connection and access to authorized hardware devices for dashboard monitoring

4.3. External Dependencies:

Tools	*Purpose*	*Risks*
Z Ai GLM API	Core reasoning engine for multi-step triage and orchestration.	High: Rate limits could cause the core product to fail; requires fallback states.
Hospital CRM	Access to existing patient records for context.	Medium: Potential downtime in external databases.

© UMHackathon 2026

Email: umhackathon@um.edu.my

5. *Project Management & Team Contributions*

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

Project Timeline:

- Day 1-3: Project Planning, PRD finalization, and Architecture design.
- Day 4-7: Backend development, GLM prompt engineering, and API integration.
- Day 8-10: Frontend development, QA Testing, and Final Documentation sign-off.

5.1. Team Members Role:

- *Noorilyamarsha Binti Elias Sabar : Team Lead & Logic Implementation (Java/OOP Architect)*
- *Kamilia Ahlam Taqi Binti Taqi Mohamed : Visual Concept Designer & Interface Designer*
- Nur Auni Faqihah Binti Fakhrurazi*: Backend API & GLM Integration*

5.2. *Recommendations:*

- Redis Cache : frequently accessed clinical guidelines and common medical codes to reduce GLM inference costs and latency

© UMHackathon 2026

Email: umhackathon@um.edu.my